Python Programming

Handwritten Notes

Basic to Advanced + Industry Programs

```
name = "Sahil"
for i in range(3):
    print(f"Hello {name}, day {i+1}")

# Python is easy!
```

Easy language | Examples with output | MCQs after every chapter
Practice programs | Interview questions | Cheat sheet

Notes by
Sahil

Index

What is inside these notes

Chapter 1 Introduction to Python 3

Chapter 2 Variables, Data Types and Operators 7

Chapter 3 Input, Output and String Formatting 12

Chapter 4 Control Flow: Decisions and Loops 16

Chapter 5 Strings 21

Chapter 6 Lists and Tuples 25

Chapter 7 Dictionary and Set 29

Chapter 8 Functions 33

Chapter 9 Modules, Packages and Virtual Environments 38

Chapter 10 File Handling and Exception Handling 42

Chapter 11 Object Oriented Programming (OOP) 47

Chapter 12 Advanced OOP: Magic Methods and Dataclasses 52

Chapter 13 Iterators, Generators and Decorators 56

Chapter 14 Working with Data: JSON, CSV, Dates and Database . . 60

Chapter 15 Advanced Python for Industry 64

Chapter 16 Data Structures and Algorithms in Python 69

Chapter 17 NumPy and Matplotlib Basics 74

Chapter 18 Industry Oriented Programs 77

Chapter 19 Practice Programs 83

Chapter 20 Interview Questions and Answers 86

Special Section Quick Revision Cheat Sheet 88

Chapter 1

Introduction to Python

1.1 What is Python?

Python is a **high level, interpreted, general purpose programming language**. It was created by **Guido van Rossum** and first released in **1991**. The name comes from the comedy show "Monty Python", not from the snake.

- **Easy to read and write** - code looks like simple English.
- **Interpreted** - runs line by line, no separate compile step.
- **Dynamically typed** - you do not declare variable types.
- **Free and open source**, works on Windows, Linux and Mac.
- **Huge libraries** - for web, data, AI, automation, games and more.
- **Multi-paradigm** - supports procedural, object oriented and functional style.

1.2 Short history

Year	Version	Note
1991	Python 0.9	First release
2000	Python 2.0	Lists comprehension, garbage collection
2008	Python 3.0	Major clean-up, not compatible with 2
2020	Python 2 ended	No more updates for Python 2
Today	Python 3.12 / 3.13	Use the latest Python 3

1.3 Where is Python used? (Industry)

Field	Use	Popular tools
Data Science / AI	analysis, machine learning	NumPy, Pandas, scikit-learn, PyTorch
Web development	websites, REST APIs	Django, Flask, FastAPI
Automation / scripting	file work, reports, testing	os, pathlib, Selenium
DevOps / Cloud	deployment scripts, tools	Ansible, boto3 (AWS)
Cyber security	scanners, analysis	scapy, requests

Field	Use	Popular tools
Embedded / IoT	Raspberry Pi, MicroPython	gpiozero, pyserial
Finance / Science	models, simulation	SciPy, Matplotlib

1.4 Installing and running Python

- Download from **python.org** and tick "Add Python to PATH".
- Check: open terminal and type `python --version`.
- Editors: **VS Code**, PyCharm, IDLE, Jupyter Notebook.

```
python hello.py          # run a file
python                   # open interactive mode (REPL)
>>> 2 + 3
5
>>> exit()
pip install requests     # install a library
```

1.5 First program

```
print("Hello, World!")
```

```
Output:
Hello, World!
```

Compare with C: no `#include`, no `main()`, no semicolon needed!

1.6 How Python runs a program

- Your `.py` file is converted to **bytecode** (`.pyc`) by the interpreter.
- The **Python Virtual Machine (PVM)** runs the bytecode line by line.

1.7 Comments and indentation

```
# This is a single line comment

"""
This is a multi line comment
(also used as a docstring)
"""

if 5 > 2:
    print("Indentation = 4 spaces")   # this block is inside the if
    print("Same block")
```

Careful: Python uses **indentation instead of curly braces**. Wrong indentation gives `IndentationError`. Use 4 spaces and never mix tabs with spaces.

1.8 Rules for naming (identifiers)

- Letters, digits and underscore only; cannot start with a digit.

- Case sensitive: `Age` and `age` are different.
- Cannot use keywords like `if`, `for`, `class`.
- Style: `snake_case` for variables/functions, `CamelCase` for classes, `UPPER_CASE` for constants.

```
import keyword
print(keyword.kwlist)      # list of all 35 keywords
```

1.9 The Zen of Python

Type `import this` in Python. Main ideas: "Beautiful is better than ugly", "Simple is better than complex", "Readability counts".

1.10 Types of errors

Error	Meaning	Example
SyntaxError	wrong grammar	`print("hi"`
IndentationError	wrong spaces	unexpected indent
NameError	variable not defined	`print(x)` before `x = 1`
TypeError	wrong type used	`"a" + 5`
ZeroDivisionError	divide by zero	`5 / 0`

MCQs - Introduction to Python

Q1. Who created Python?

(a) Dennis Ritchie
(b) Guido van Rossum
(c) James Gosling
(d) Bjarne Stroustrup

Q2. Python is

(a) compiled only
(b) interpreted
(c) a markup language
(d) a database

Q3. Which symbol starts a single line comment?

(a) //
(b) /*
(c) #
(d) --

Q4. Python blocks are defined by

(a) curly braces
(b) indentation
(c) semicolons
(d) begin / end

Q5. Which is a valid variable name?

(a) 2name
(b) my-name
(c) my_name
(d) class

Q6. Extension of a Python file is

(a) .pt
(b) .pyt
(c) .py
(d) .python

Q7. `print("Hi")` **is a**

(a) function call
(b) keyword
(c) variable
(d) module

Q8. Python is

(a) case sensitive
(b) not case sensitive
(c) only upper case
(d) only lower case

Q9. Which tool installs libraries?

(a) pip
(b) git
(c) make
(d) gcc

Q10. Python code is first converted into

(a) machine code
(b) bytecode
(c) assembly
(d) HTML

Answers: 1-b 2-b 3-c 4-b 5-c 6-c 7-a 8-a 9-a 10-b

Chapter 2

Variables, Data Types and Operators

2.1 Variables

A variable is a **name that refers to a value**. No need to declare the type; Python decides it from the value.

```
age = 20              # int
price = 99.5          # float
name = "Sahil"        # str
is_student = True     # bool
x = y = z = 0         # same value to many
a, b = 10, 20         # multiple assignment
a, b = b, a           # swap in one line!
```

Remember: Python variables are like **labels on boxes** - the same label can later be moved to a box of another type: `x = 5` then `x = "five"` is allowed.

2.2 Basic data types

Type	Meaning	Example
int	whole numbers (no size limit)	10, -3, 10**100
float	decimal numbers	3.14, 2.5e3
complex	real + imaginary	3 + 4j
str	text	"hello", 'hi'
bool	True or False	True, False
NoneType	no value	None
list	ordered, changeable	[1, 2, 3]
tuple	ordered, fixed	(1, 2, 3)
set	unique items, no order	{1, 2, 3}
dict	key : value pairs	{"a": 1}

```
print(type(10))        # <class 'int'>
print(type(3.5))       # <class 'float'>
print(type("hi"))      # <class 'str'>
print(isinstance(5, int))   # True
```

2.3 Type conversion (casting)

```
int("25")        # 25
float("3.5")     # 3.5
str(100)         # "100"
bool(0)          # False   (0, "", [], None are False)
bool("hi")       # True
list("abc")      # ['a', 'b', 'c']
int(3.9)         # 3  (cuts decimal, no rounding)
round(3.567, 2)  # 3.57
```

2.4 Mutable and immutable

- **Immutable** (cannot change after creation): int, float, str, tuple, bool, frozenset.
- **Mutable** (can change): list, dict, set.

```
s = "hello"
# s[0] = "H"        # TypeError: strings are immutable
s = "H" + s[1:]     # make a new string instead
```

2.5 Operators

Arithmetic operators

Operator	Name	Example	Result
+	add	7 + 2	9
-	subtract	7 - 2	5
*	multiply	7 * 2	14
/	true division	7 / 2	3.5
//	floor division	7 // 2	3
%	remainder	7 % 2	1
**	power	7 ** 2	49

Remember: In Python / always gives a **float**. Use // when you want integer division.

Comparison, logical, assignment

Type	Operators
Comparison	== != > < >= <= (can chain: 1 < x < 10)
Logical	and, or, not
Assignment	= += -= *= /= //= %= **=
Identity	is, is not (same object in memory?)
Membership	in, not in (is item present?)
Bitwise	& | ^ ~ << >>

```
print(5 > 3 and 2 > 1)       # True
print(3 in [1, 2, 3])        # True
print("a" not in "hello")    # True
x = [1]; y = [1]
print(x == y, x is y)        # True False (equal values, different objects)
print(10 if 5 > 3 else 20)   # ternary: 10
```

Walrus operator (Python 3.8+)

```
if (n := len("python")) > 3:
    print(n)          # 6   assigns and uses in one line
```

2.6 Operator precedence (high to low)

() -> ** -> +x -x ~x -> * / // % -> + - -> << >> -> & -> ^ -> | -> comparisons -> not -> and -> or

2.7 Numbers and math

```
import math
print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.141592653589793
print(math.ceil(2.1), math.floor(2.9))   # 3 2
print(abs(-7), pow(2, 5), divmod(17, 5)) # 7 32 (3, 2)
print(max(3, 9, 4), min(3, 9, 4), sum([1, 2, 3]))
print(0.1 + 0.2)          # 0.30000000000000004  (float is not exact)
print(round(0.1 + 0.2, 2))# 0.3
```

2.8 Constants and the id() function

Python has no real constants. By rule we write them in UPPER CASE and do not change them: `PI = 3.14`, `MAX_USERS = 100`.

```
a = 10
b = a
print(id(a) == id(b))   # True - both labels point to the same object
```

MCQs - Variables, Data Types and Operators

Q1. Which is NOT a Python data type?

(a) list
(b) tuple
(c) array[] declared as int
(d) dict

Q2. `7 // 2` gives

(a) 3.5
(b) 3
(c) 4
(d) 3.0

Q3. `2 ** 3` **gives**

(a) 6
(b) 5
(c) 8
(d) 9

Q4. `type(3.0)` **is**

(a) int
(b) float
(c) str
(d) double

Q5. Which one is immutable?

(a) list
(b) dict
(c) tuple
(d) set

Q6. `bool("")` **is**

(a) True
(b) False
(c) None
(d) Error

Q7. `int("12") + 3` **is**

(a) 15
(b) 123
(c) "123"
(d) Error

Q8. `10 % 3` **is**

(a) 3
(b) 1
(c) 0
(d) 3.33

Q9. `"a" in "cat"` **is**

(a) True
(b) False
(c) Error
(d) None

Q10. `5 / 2` **gives**

(a) 2
(b) 2.5
(c) 3
(d) 2.0 only in Python 2

Q11. Which keyword is used for "and" logic?

(a) &&
(b) and
(c) AND only
(d) &

Answers: 1-c 2-b 3-c 4-b 5-c 6-b 7-a 8-b 9-a 10-b 11-b

Chapter 3

Input, Output and String Formatting

3.1 Output with print()

```
print("Hello")
print("Sahil", 20, True)              # items separated by space
print("a", "b", sep="-")              # a-b
print("Loading", end="...")           # no new line at the end
print("Done")                         # Loading...Done
print("Line1\nLine2\tTabbed")
```

3.2 Input with input()

`input()` *always returns a* ***string****. Convert it when you need a number.*

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter height in feet: "))
print("Hello", name, "you are", age, "years old")

a, b = map(int, input("Enter two numbers: ").split())
print("Sum =", a + b)
```

```
Output:
Enter your name: Sahil
Enter your age: 20
Enter height in feet: 5.9
Hello Sahil you are 20 years old
Enter two numbers: 4 6
Sum = 10
```

3.3 String formatting (3 ways)

```
name, marks = "Sahil", 88.456

print("Name: %s, Marks: %.1f" % (name, marks))      # 1. old style
print("Name: {}, Marks: {:.1f}".format(name, marks))# 2. format()
print(f"Name: {name}, Marks: {marks:.1f}")          # 3. f-string (best)
```

f-string tricks

```
n = 42
print(f"{n:5d}|")        # '   42|' width 5
print(f"{n:<5d}|")       # '42   |' left align
print(f"{n:05d}")        # 00042
print(f"{1234567:,}")    # 1,234,567
print(f"{0.256:.1%}")    # 25.6%
print(f"{n:b} {n:x} {n:o}")   # 101010 2a 52  (binary, hex, octal)
print(f"{n * 2 = }")     # n * 2 = 84   (debug style)
```

3.4 Simple programs

(1) Area and perimeter of a rectangle

```
l = float(input("Length: "))
w = float(input("Width: "))
print(f"Area = {l * w}, Perimeter = {2 * (l + w)}")
```

(2) Simple interest

```
p, r, t = 10000, 7.5, 2
print(f"Simple Interest = {p * r * t / 100:.2f}")    # 1500.00
```

(3) Swap two numbers

```
a, b = 10, 20
a, b = b, a
print(a, b)     # 20 10
```

(4) Sum of digits

```
n = 1234
print(sum(int(d) for d in str(n)))    # 10
```

3.5 Command line arguments and exit

```
import sys
print(sys.argv)          # list of arguments, argv[0] is the script name
if len(sys.argv) < 2:
    sys.exit("Usage: python app.py <name>")
```

3.6 Escape sequences and raw strings

```
print("She said \"Hi\"")     # She said "Hi"
print('It\'s fine')          # It's fine
print(r"C:\new\test")        # raw string: backslashes stay as they are
print("""Triple quotes
allow many lines""")
```

> **Industry use:** Real programs never trust input: always validate (check type, range, empty values) before using it.

MCQs - Input, Output and String Formatting

Q1. `input()` **returns data of type**

(a) int
(b) float
(c) str
(d) bool

Q2. To read an integer we write

(a) int(input())
(b) input(int)
(c) scanf()
(d) cin

Q3. `print("a", "b", sep="-")` **prints**

(a) a b
(b) a-b
(c) ab
(d) a,b

Q4. `end=""` **in print() will**

(a) stop program
(b) not add new line
(c) add 2 lines
(d) clear screen

Q5. Which is an f-string?

(a) "Hi {name}"
(b) f"Hi {name}"
(c) "Hi" % name
(d) format"Hi"

Q6. `f"{3.14159:.2f}"` **gives**

(a) 3.1
(b) 3.14
(c) 3.142
(d) 3.15

Q7. A raw string starts with

(a) r
(b) b
(c) u
(d) w

Q8. `\n` **means**

(a) tab
(b) new line
(c) space
(d) backslash

Q9. `sys.argv[0]` **is**

(a) first argument
(b) script name
(c) count
(d) Python path

Q10. `a, b = b, a` **will**

(a) give error
(b) swap values
(c) copy a
(d) delete b

Answers: 1-c 2-a 3-b 4-b 5-b 6-b 7-a 8-b 9-b 10-b

Chapter 4

Control Flow: Decisions and Loops

4.1 if, elif, else

```
marks = int(input("Enter marks: "))

if marks >= 90:
    print("Grade A+")
elif marks >= 75:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Fail")
```

```
Output:
Enter marks: 82
Grade A
```

Remember: These values are **False**: `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`, `None`. Everything else is **True**.

Nested if, one line if, and match-case

```
age = 20
status = "adult" if age >= 18 else "minor"      # ternary

day = 3
match day:                          # Python 3.10+ (like switch)
    case 1:
        print("Monday")
    case 2 | 3:
        print("Tuesday or Wednesday")
    case _:
        print("Other day")          # _ is default
```

4.2 while loop

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1
# 1 2 3 4 5

while True:                         # infinite loop with break
    cmd = input("Type quit to exit: ")
    if cmd == "quit":
        break
```

4.3 for loop and range()

A `for` loop goes through each item of a sequence (list, string, range, dictionary ...).

```
for ch in "Hi!":
    print(ch)

for i in range(5):              # 0 1 2 3 4
    print(i, end=" ")
for i in range(1, 11, 2):       # start, stop(not included), step -> 1 3 5 7 9
    print(i, end=" ")
for i in range(10, 0, -3):      # 10 7 4 1
    print(i, end=" ")

fruits = ["apple", "banana", "mango"]
for index, fruit in enumerate(fruits, start=1):
    print(index, fruit)
```

4.4 break, continue, pass, else with loops

```
for n in range(1, 10):
    if n == 3:
        continue          # skip 3
    if n == 7:
        break             # stop at 7
    print(n, end=" ")     # 1 2 4 5 6

for n in range(2, 6):
    if n == 10:
        break
else:
    print("loop finished without break")   # runs when no break happened

def todo():
    pass                  # empty block placeholder
```

4.5 Nested loops and patterns

```
for i in range(1, 5):
    print("* " * i)
```

```
Output:
*
* *
* * *
* * * *
```

```
rows = 4
for i in range(1, rows + 1):
    print(" " * (rows - i) + " ".join(str(j) for j in range(1, i + 1)))
```

```
Output:
   1
  1 2
 1 2 3
1 2 3 4
```

4.6 Important programs

(1) Factorial

```
n, f = 5, 1
for i in range(2, n + 1):
    f *= i
print(f)          # 120   (or: import math; math.factorial(5))
```

(2) Fibonacci

```
a, b = 0, 1
for _ in range(10):
    print(a, end=" ")
    a, b = b, a + b
# 0 1 1 2 3 5 8 13 21 34
```

(3) Prime numbers

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

print([x for x in range(1, 30) if is_prime(x)])
# [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

(4) Palindrome and Armstrong

```
s = "madam"
print(s == s[::-1])                                  # True

n = 153
print(n == sum(int(d) ** len(str(n)) for d in str(n)))   # True
```

(5) Number guessing game

```
import random
secret = random.randint(1, 20)
for attempt in range(1, 6):
    guess = int(input(f"Attempt {attempt}/5 - guess: "))
    if guess == secret:
        print("Correct!"); break
    print("Too low" if guess < secret else "Too high")
else:
    print("You lost. Number was", secret)
```

MCQs - Control Flow: Decisions and Loops

Q1. Which keyword is used for "else if" in Python?

(a) elseif
(b) else if
(c) elif
(d) elsif

Q2. `range(1, 5)` **produces**

(a) 1 2 3 4 5
(b) 1 2 3 4
(c) 0 1 2 3 4
(d) 2 3 4

Q3. `range(10, 0, -5)` **gives**

(a) 10 5
(b) 10 5 0
(c) 5 0
(d) error

Q4. `continue` **does**

(a) ends loop
(b) skips to next iteration
(c) ends program
(d) nothing

Q5. `pass` **statement**

(a) stops the loop
(b) does nothing (placeholder)
(c) returns value
(d) raises error

Q6. `else` **block of a loop runs when**

(a) loop has break
(b) loop finishes without break
(c) always
(d) never

Q7. `match-case` **is available from**

(a) 2.7
(b) 3.6
(c) 3.10
(d) 1.0

Q8. Which of these is False?

(a) "0"
(b) []
(c) [0]
(d) " "

Q9. `for i in "abc": print(i)` **prints**

(a) abc in one line
(b) a b c on separate lines
(c) 0 1 2
(d) error

Q10. `enumerate(["a","b"])` **gives pairs**

(a) (0,"a"), (1,"b")
(b) ("a",0)
(c) (1,"a")
(d) only indexes

Answers: 1-c 2-b 3-a 4-b 5-b 6-b 7-c 8-b 9-b 10-a

Chapter 5

Strings

5.1 What is a string?

A string is a **sequence of characters** in quotes. It is **immutable**. Indexing starts from 0 and negative index counts from the end.

```
s = "Python"
#    P  y  t  h  o  n
#    0  1  2  3  4  5
#   -6 -5 -4 -3 -2 -1
print(s[0], s[-1])      # P n
print(s[1:4])           # yth   (start included, end not included)
print(s[:2], s[2:])     # Py thon
print(s[::2])           # Pto   (every 2nd char)
print(s[::-1])          # nohtyP (reverse)
print(len(s))           # 6
```

5.2 Important string methods

Method	Job	Example
upper() / lower()	change case	"Hi".upper() -> "HI"
title() / capitalize()	first letter capital	"hello world".title()
strip() / lstrip() / rstrip()	remove spaces at ends	" hi ".strip() -> "hi"
split(sep)	break into list	"a,b,c".split(",")
join(list)	list into string	"-".join(["a","b"]) -> "a-b"
replace(old, new)	replace text	"cat".replace("c","b")
find(sub) / index(sub)	position of sub (find gives -1 if missing)	"hello".find("l") -> 2
count(sub)	how many times	"banana".count("a") -> 3
startswith() / endswith()	check begin / end	"file.py".endswith(".py")
isdigit() / isalpha() / isalnum()	check content	"123".isdigit()
zfill(n) / center(n)	pad	"7".zfill(3) -> "007"

5.3 String operations

```
a, b = "Good", "Morning"
print(a + " " + b)        # concatenation
print(a * 3)              # GoodGoodGood
print("o" in a)           # True
print(a < b)              # True (dictionary order)
for ch in a:              # loop through letters
    print(ch, end=",")
```

5.4 Programs on strings

(1) Count vowels

```
text = "Programming Language"
vowels = sum(1 for ch in text.lower() if ch in "aeiou")
print(vowels)     # 7
```

(2) Palindrome ignoring case and spaces

```
def is_pal(s):
    t = "".join(ch.lower() for ch in s if ch.isalnum())
    return t == t[::-1]
print(is_pal("A man a plan a canal Panama"))   # True
```

(3) Word frequency

```
sentence = "to be or not to be"
freq = {}
for w in sentence.split():
    freq[w] = freq.get(w, 0) + 1
print(freq)     # {'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

(4) Anagram check

```
def is_anagram(a, b):
    return sorted(a.lower()) == sorted(b.lower())
print(is_anagram("listen", "silent"))     # True
```

(5) Caesar cipher (simple encryption)

```
def caesar(text, shift):
    out = ""
    for ch in text:
        if ch.isalpha():
            base = ord("A") if ch.isupper() else ord("a")
            out += chr((ord(ch) - base + shift) % 26 + base)
        else:
            out += ch
    return out
print(caesar("Hello, World", 3))     # Khoor, Zruog
```

(6) Remove duplicate characters keeping order

```
print("".join(dict.fromkeys("programming")))     # progamin
```

5.5 Regular expressions (pattern matching)

```
import re
text = "Call 98765-43210 or mail sahil@example.com"
print(re.findall(r"\d{5}-\d{5}", text))                 # ['98765-43210']
print(re.search(r"[\w.]+@[\w.]+\.\w+", text).group())   # sahil@example.com
print(re.sub(r"\d", "#", "a1b22"))                      # a#b##
```

Pattern	Meaning	Pattern	Meaning
.	any character	\d	digit
\w	letter, digit, _	\s	space
*	0 or more	+	1 or more
?	0 or 1	{n}	exactly n times
^	start	$	end
[abc]	one of a, b, c	( )	group

5.6 Useful extras

```
print(ord("A"), chr(97))         # 65 a
print("Hello"[::-1])             # olleH
print(" ".join(reversed("one two three".split())))   # three two one
print("abc".encode())            # b'abc'   (str to bytes)
```

MCQs - Strings

Q1. `"Python"[1:4]` **gives**

(a) Pyth
(b) yth
(c) ytho
(d) tho

Q2. `"Hello"[-1]` **is**

(a) H
(b) o
(c) l
(d) error

Q3. Strings in Python are

(a) mutable
(b) immutable
(c) arrays of int
(d) numbers

Q4. `"a,b,c".split(",")` **returns**

(a) "abc"
(b) ["a","b","c"]
(c) ("a","b","c")
(d) {a,b,c}

Q5. `"-".join(["x","y"])` **gives**

(a) x-y
(b) xy
(c) -xy
(d) x y

Q6. `"banana".count("a")` **is**

(a) 2
(b) 3
(c) 4
(d) 1

Q7. `" hi ".strip()` **returns**

(a) " hi"
(b) "hi "
(c) "hi"
(d) " hi "

Q8. `"abc"[::-1]` **gives**

(a) abc
(b) cba
(c) bca
(d) error

Q9. `"Hi" * 2` **gives**

(a) HiHi
(b) Hi2
(c) error
(d) Hi Hi

Q10. Which module is used for regular expressions?

(a) regex
(b) re
(c) string
(d) pattern

Answers: 1-b 2-b 3-b 4-b 5-a 6-b 7-c 8-b 9-a 10-b

Chapter 6

Lists and Tuples

6.1 List

A list is an **ordered, changeable (mutable)** collection. It can hold items of different types and allows duplicates.

```
nums = [10, 20, 30, 40]
mixed = [1, "two", 3.0, [4, 5]]
empty = []
print(nums[0], nums[-1])      # 10 40
nums[1] = 25                  # change
print(nums[1:3])              # [25, 30]
print(len(nums))              # 4
```

List methods

Method	Job
append(x)	add at end
insert(i, x)	add at position i
extend(list2)	add many items
remove(x)	delete first x
pop(i)	remove and return item (last if no i)
index(x) / count(x)	position / how many
sort() / sorted(list)	sort in place / return new sorted list
reverse()	reverse in place
copy() / clear()	copy / empty

```
a = [3, 1, 2]
a.append(4)            # [3, 1, 2, 4]
a.insert(0, 9)         # [9, 3, 1, 2, 4]
a.remove(1)            # [9, 3, 2, 4]
print(a.pop())         # 4
a.sort(reverse=True)   # [9, 3, 2]
print(sum(a), max(a), min(a))
del a[0]               # delete by index
```

6.2 Looping and list comprehension

```
squares = [x * x for x in range(1, 6)]           # [1, 4, 9, 16, 25]
evens   = [x for x in range(20) if x % 2 == 0]
pairs   = [(a, b) for a in (1, 2) for b in "xy"]  # [(1,'x'),(1,'y'),(2,'x'),(2,'y')]
upper   = [w.upper() for w in ["a", "b"]]
matrix  = [[1, 2, 3], [4, 5, 6]]
flat    = [n for row in matrix for n in row]      # [1,2,3,4,5,6]
```

6.3 Copy trap

```
a = [1, 2, 3]
b = a            # same list, two names!
b.append(4)
print(a)         # [1, 2, 3, 4]  a also changed

c = a.copy()     # real copy (or a[:] or list(a))
import copy
d = copy.deepcopy([[1], [2]])   # copy of nested lists too
```

6.4 Programs on lists

(1) Second largest

```
nums = [12, 45, 7, 89, 23]
print(sorted(set(nums))[-2])     # 45
```

(2) Remove duplicates keeping order

```
data = [1, 2, 2, 3, 1, 4]
print(list(dict.fromkeys(data)))    # [1, 2, 3, 4]
```

(3) Rotate list left by k

```
a, k = [1, 2, 3, 4, 5], 2
print(a[k:] + a[:k])           # [3, 4, 5, 1, 2]
```

(4) Binary search

```
def bsearch(a, key):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == key: return mid
        if a[mid] < key:  lo = mid + 1
        else:             hi = mid - 1
    return -1
print(bsearch([1, 3, 5, 7, 9], 7))     # 3
```

(5) Bubble sort

```
def bubble(a):
    n = len(a)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a
print(bubble([5, 1, 4, 2, 8]))     # [1, 2, 4, 5, 8]
```

(6) Matrix transpose

```
m = [[1, 2, 3], [4, 5, 6]]
print([list(r) for r in zip(*m)])     # [[1, 4], [2, 5], [3, 6]]
```

6.5 Tuple

A tuple is **ordered but cannot be changed (immutable)**. Faster than list and safe for fixed data (coordinates, RGB colour, database rows).

```
t = (10, 20, 30)
single = (5,)              # comma needed for one item
x, y, z = t                # unpacking
first, *rest = t           # first=10, rest=[20, 30]
print(t.count(10), t.index(20))
# t[0] = 5                 # TypeError
```

List	Tuple
[ ] mutable	( ) immutable
slower, more memory	faster, less memory
can be changed, sorted	fixed data, can be dictionary key

MCQs - Lists and Tuples

Q1. Lists are written with

(a) ( )
(b) { }
(c) [ ]
(d) < >

Q2. `a = [1,2,3]; a.append(4)` **gives**

(a) [1,2,3,4]
(b) [4,1,2,3]
(c) error
(d) [1,2,3]

Q3. `[1,2,3][-1]` **is**

(a) 1
(b) 3
(c) error
(d) 2

Q4. Which method removes and returns the last item?

(a) remove()
(b) delete()
(c) pop()
(d) cut()

Q5. `[x*2 for x in range(3)]` **is**

(a) [0,2,4]
(b) [2,4,6]
(c) [0,1,2]
(d) [1,2,3]

Q6. Tuples are

(a) mutable
(b) immutable
(c) unordered
(d) key value pairs

Q7. A tuple with one item is written as

(a) (5)
(b) (5,)
(c) [5]
(d) {5}

Q8. `b = a` **for a list makes**

(a) a copy
(b) another name for same list
(c) an error
(d) a tuple

Q9. `sorted([3,1,2])` **returns**

(a) None
(b) [1,2,3]
(c) [3,2,1]
(d) sorts original

Q10. `len([1,[2,3]])` **is**

(a) 3
(b) 2
(c) 1
(d) 4

Answers: 1-c 2-a 3-b 4-c 5-a 6-b 7-b 8-b 9-b 10-b

Chapter 7

Dictionary and Set

7.1 Dictionary (dict)

A dictionary stores **key : value** pairs. Keys must be unique and immutable (str, int, tuple). Since Python 3.7 it keeps the **insertion order**. Lookup is very fast (hash table).

```
student = {"name": "Sahil", "age": 20, "marks": [80, 90]}
print(student["name"])            # Sahil
print(student.get("city", "NA"))  # NA  (safe: no error if key missing)
student["age"] = 21               # change
student["city"] = "Solapur"       # add new
del student["marks"]              # delete
print("name" in student)          # True (checks keys)
print(len(student))
```

Dictionary methods

Method	Job
keys() / values() / items()	all keys / all values / all (key, value) pairs
get(k, default)	safe read
pop(k) / popitem()	remove key / remove last pair
update(d2)	merge another dictionary
setdefault(k, v)	add key if missing, return value
copy() / clear()	copy / empty

```
d = {"a": 1, "b": 2}
for k, v in d.items():
    print(k, "->", v)
squares = {n: n * n for n in range(1, 5)}     # {1:1, 2:4, 3:9, 4:16}
merged = {**d, "c": 3}                        # {'a':1,'b':2,'c':3}
merged2 = d | {"z": 26}                       # Python 3.9+
sorted_d = dict(sorted(d.items(), key=lambda kv: kv[1], reverse=True))
```

7.2 Nested dictionary and JSON-like data

```
company = {
    "name": "TechSoft",
    "employees": [
        {"id": 1, "name": "Ravi", "skills": ["C", "Python"]},
        {"id": 2, "name": "Neha", "skills": ["SQL"]},
    ],
}
print(company["employees"][0]["skills"][1])     # Python
for e in company["employees"]:
    print(e["name"], len(e["skills"]))
```

7.3 Programs with dictionaries

(1) Character frequency using Counter

```
from collections import Counter
print(Counter("mississippi"))                  # Counter({'i':4,'s':4,'p':2,'m':1})
print(Counter("mississippi").most_common(2))   # [('i', 4), ('s', 4)]
```

(2) Group words by length

```
from collections import defaultdict
groups = defaultdict(list)
for w in ["cat", "door", "dog", "tree", "sun"]:
    groups[len(w)].append(w)
print(dict(groups))     # {3: ['cat','dog','sun'], 4: ['door','tree']}
```

(3) Two sum problem (interview favourite)

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
print(two_sum([2, 7, 11, 15], 9))      # (0, 1)
```

7.4 Set

*A set is an **unordered collection of unique items**. Good for removing duplicates and for maths operations.*

```
s = {1, 2, 3, 3, 2}          # {1, 2, 3}
s.add(4); s.discard(10)      # discard does not give error if missing
empty = set()                # {} makes an empty dict, not set!
print(list(set([1, 1, 2, 3, 3])))       # remove duplicates
```

Operation	Symbol	Example (A={1,2,3}, B={3,4})
Union	A | B	{1,2,3,4}
Intersection	A & B	{3}
Difference	A - B	{1,2}
Symmetric difference	A ^ B	{1,2,4}
Subset check	A <= B	False

```
a, b = {1, 2, 3}, {3, 4}
print(a | b, a & b, a - b, a ^ b)
common = set("python") & set("typhoon")
fs = frozenset([1, 2])      # immutable set
```

7.5 Choosing the right collection

Need	Use
ordered, changeable	list
fixed data	tuple
unique items, fast membership test	set
label -> value lookup	dict

Industry use: In companies, API data comes as JSON which becomes **dict and list** in Python. Counting, grouping and lookup with dictionaries is used in almost every data script.

MCQs - Dictionary and Set

Q1. Dictionaries store data as

(a) index : value
(b) key : value
(c) value only
(d) tuple only

Q2. `d.get("x", 0)` **when "x" is missing returns**

(a) KeyError
(b) None
(c) 0
(d) "x"

Q3. Dictionary keys must be

(a) mutable
(b) unique and immutable
(c) numbers only
(d) lists

Q4. `{}` **creates an empty**

(a) set
(b) list
(c) dict
(d) tuple

Q5. `{1,2,2,3}` **equals**

(a) {1,2,2,3}
(b) {1,2,3}
(c) [1,2,3]
(d) error

Q6. `{1,2} | {2,3}` **gives**

(a) {2}
(b) {1,2,3}
(c) {1,3}
(d) {1}

Q7. `{1,2,3} & {2,3,4}` **gives**

(a) {1,4}
(b) {2,3}
(c) {1,2,3,4}
(d) {}

Q8. Which loop gives key and value?

(a) d.keys()
(b) d.values()
(c) d.items()
(d) d.get()

Q9. Sets are

(a) ordered
(b) unordered with unique items
(c) mutable lists
(d) key value

Q10. Which module gives Counter?

(a) itertools
(b) collections
(c) math
(d) random

Answers: 1-b 2-c 3-b 4-c 5-b 6-b 7-b 8-c 9-b 10-b

Chapter 8

Functions

8.1 What is a function?

A function is a **named block of code** that does one job and can be reused. Defined with `def`.

```
def greet(name):                    # name is a parameter
    """Return a greeting."""        # docstring
    return f"Hello, {name}!"

msg = greet("Sahil")                # "Sahil" is an argument
print(msg)                          # Hello, Sahil!
help(greet)                         # shows the docstring
```

8.2 Types of arguments

```
def info(name, age=18, *hobbies, **extra):
    print(name, age, hobbies, extra)

info("Ravi")                                # default value used
info("Neha", 22)                            # positional
info(age=25, name="Amit")                   # keyword arguments (any order)
info("Sahil", 20, "chess", "coding", city="Pune", job="dev")
# Sahil 20 ('chess','coding') {'city': 'Pune', 'job': 'dev'}
```

Kind	Meaning
Positional	matched by position
Keyword	matched by name: `f(x=1)`
Default	`def f(a, b=5)`
`*args`	any number of extra positional args (tuple)
`**kwargs`	any number of extra keyword args (dict)
Positional-only `/`, keyword-only `*`	`def f(a, /, b, *, c)`

Careful: Never use a mutable default like `def f(x, items=[])`. The same list is shared between calls! Use `items=None` and create the list inside.

```
def add_item(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
```

8.3 Return values

```
def stats(nums):
    return min(nums), max(nums), sum(nums) / len(nums)    # returns a tuple

lo, hi, avg = stats([4, 8, 6])
print(lo, hi, avg)       # 4 8 6.0
```

8.4 Scope: LEGB rule

*Python looks for a name in this order: **L**ocal -> **E**nclosing -> **G**lobal -> **B**uilt-in.*

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        nonlocal x           # use outer's x
        x = "changed"
    inner()
    print(x)                 # changed

count = 0
def inc():
    global count             # change the global variable
    count += 1
```

8.5 Lambda, map, filter, reduce

```
square = lambda x: x * x                  # small unnamed function
print(square(5))                          # 25

nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda x: x * 2, nums)))           # [2,4,6,8,10,12]
print(list(filter(lambda x: x % 2 == 0, nums)))   # [2,4,6]
from functools import reduce
print(reduce(lambda a, b: a * b, nums))           # 720

people = [("Ravi", 25), ("Neha", 22), ("Amit", 30)]
people.sort(key=lambda p: p[1])           # sort by age
```

8.6 Recursion

```
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)

def fib(n, memo={}):                # memoization makes it fast
    if n < 2: return n
    if n not in memo:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]

print(factorial(5), fib(50))        # 120 12586269025
import sys
print(sys.getrecursionlimit())      # 1000 by default
```

More recursion programs

```
def power(b, e):    return 1 if e == 0 else b * power(b, e - 1)
def sum_digits(n):  return 0 if n == 0 else n % 10 + sum_digits(n // 10)
def hanoi(n, a, c, b):
    if n:
        hanoi(n - 1, a, b, c)
        print(f"Move disc {n} from {a} to {c}")
        hanoi(n - 1, b, c, a)
```

8.7 Type hints (good practice in industry)

```
def area(width: float, height: float) -> float:
    return width * height

from typing import Optional
def find(name: str, items: list[str]) -> Optional[int]:
    return items.index(name) if name in items else None
```

> **Remember:** Type hints are **not enforced** by Python, but tools like `mypy` and editors use them to find bugs early.

8.8 Useful built-in functions

`len, range, sum, min, max, sorted, reversed, enumerate, zip, map, filter, any, all, abs, round, isinstance, type, input, print, open`

```
names = ["a", "b"]; ages = [10, 20]
print(list(zip(names, ages)))        # [('a', 10), ('b', 20)]
print(any([0, 0, 5]), all([1, 1, 0]))   # True False
```

MCQs - Functions

Q1. Functions are defined with the keyword

(a) function
(b) def
(c) fun
(d) define

Q2. `*args` collects arguments into a

(a) list
(b) tuple
(c) dict
(d) set

Q3. `**kwargs` **collects arguments into a**

(a) tuple
(b) list
(c) dict
(d) string

Q4. A function without `return` **gives**

(a) 0
(b) None
(c) error
(d) ""

Q5. `lambda x: x+1` **is**

(a) a loop
(b) an anonymous function
(c) a class
(d) a module

Q6. LEGB stands for

(a) Local, Enclosing, Global, Built-in
(b) List, Element, Group, Block
(c) Loop, Else, Goto, Break
(d) None

Q7. To change a global variable inside a function use

(a) nonlocal
(b) global
(c) static
(d) extern

Q8. `list(filter(lambda x: x>2, [1,2,3,4]))` **is**

(a) [1,2]
(b) [3,4]
(c) [2,3]
(d) [4]

Q9. Why is `def f(x, a=[])` **a bad idea?**

(a) syntax error
(b) the list is shared between calls
(c) slower
(d) not allowed

Q10. `any([0, 0, 1])` **returns**

(a) True
(b) False
(c) 1
(d) Error

Answers: 1-b 2-b 3-c 4-b 5-b 6-a 7-b 8-b 9-b 10-a

Chapter 9 Modules, Packages and Virtual Environments

9.1 Modules

A module is simply a **Python file** with functions, classes and variables that you can reuse.

```
# file: mathutil.py
PI = 3.14159
def area(r):
    return PI * r * r

if __name__ == "__main__":      # runs only when this file is run directly
    print(area(2))

# file: main.py
import mathutil
from mathutil import area
from mathutil import area as circle_area
import math as m
print(mathutil.area(3), area(3), m.sqrt(9))
```

Remember: `__name__ == "__main__"` is true only when the file is executed directly, not when it is imported. It is used in every professional script.

9.2 Packages

A package is a **folder of modules** that contains `__init__.py`.

```
myapp/
    __init__.py
    utils/
        __init__.py
        strings.py
        files.py
    main.py

# in main.py
from utils.strings import clean_text
```

9.3 Useful built-in modules

Module	Use
math, random, statistics	numbers, random choice, mean / median
datetime, time	dates and time
os, sys, pathlib, shutil	files, folders, system

Module	Use
json, csv	data files
re	regular expressions
collections, itertools, functools	special containers and tools
logging, argparse	logs and command line options
unittest, doctest	testing
threading, multiprocessing, asyncio	parallel work

```
import random, statistics, datetime
print(random.choice(["a", "b", "c"]))
print(random.sample(range(1, 50), 6))
print(statistics.mean([1, 2, 3, 4]), statistics.median([1, 3, 9]))
print(datetime.date.today())
```

9.4 pip - installing packages

```
pip install requests              # install
pip install numpy==1.26.4         # exact version
pip list                          # installed packages
pip show requests                 # details
pip freeze > requirements.txt     # save versions
pip install -r requirements.txt   # install all from file
pip uninstall requests
```

9.5 Virtual environment (venv)

Each project gets its **own separate set of packages**, so versions do not clash. Every company project uses this.

```
python -m venv venv               # create environment named "venv"
venv\Scripts\activate             # activate on Windows
source venv/bin/activate          # activate on Linux / Mac
pip install flask                 # installs only inside this venv
deactivate                        # leave
```

9.6 Popular third-party libraries

Library	Job	Quick example
requests	call web APIs	`requests.get(url).json()`
numpy	fast arrays and maths	`np.array([1,2,3]) * 2`
pandas	tables and data analysis	`pd.read_csv("a.csv")`
matplotlib	graphs	`plt.plot(x, y)`
flask / fastapi	web servers and APIs	few lines to make an API
pytest	easy testing	`def test_add(): assert add(2,3)==5`

9.7 Example: calling a web API

```
import requests
resp = requests.get("https://api.github.com/users/octocat", timeout=10)
if resp.status_code == 200:
    data = resp.json()                  # dict
    print(data["login"], data["public_repos"])
else:
    print("Error", resp.status_code)
```

MCQs - Modules, Packages and Virtual Environments

Q1. A Python module is

(a) a folder
(b) a .py file
(c) a class
(d) a variable

Q2. Command to install a library

(a) install pip
(b) pip install name
(c) python get name
(d) import name

Q3. `from math import sqrt` **lets us call**

(a) math.sqrt()
(b) sqrt()
(c) Math.sqrt()
(d) import()

Q4. Folder becomes a package when it has

(a) main.py
(b) __init__.py
(c) setup.bat
(d) config

Q5. `if __name__ == "__main__":` **runs the block when**

(a) file is imported
(b) file is run directly
(c) never
(d) always

Q6. Command to create a virtual environment

(a) python -m venv env
(b) pip venv env
(c) python env create
(d) make venv

Q7. `pip freeze > requirements.txt` **does**

(a) deletes packages
(b) saves installed package list
(c) updates Python
(d) freezes program

Q8. Which module works with dates?

(a) math
(b) datetime
(c) re
(d) json

Q9. Why use venv?

(a) faster CPU
(b) separate dependencies per project
(c) smaller Python
(d) to avoid typing

Q10. Library to call web APIs

(a) numpy
(b) requests
(c) re
(d) random

Answers: 1-b 2-b 3-b 4-b 5-b 6-a 7-b 8-b 9-b 10-b

Chapter 10

File Handling and Exception Handling

10.1 Working with text files

Use `open(filename, mode)`. The best way is the **with** statement - it closes the file automatically, even if an error happens.

Mode	Meaning
"r"	read (default), error if file missing
"w"	write, erases old content
"a"	append at the end
"x"	create new, error if exists
"r+"	read and write
"b" added (rb, wb)	binary files (images, pdf)

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Line 1\n")
    f.writelines(["Line 2\n", "Line 3\n"])

with open("notes.txt") as f:
    print(f.read())               # whole file as one string

with open("notes.txt") as f:
    for line in f:                # best way for big files: one line at a time
        print(line.strip())

with open("notes.txt", "a") as f:
    f.write("Line 4\n")
```

File methods

`read()`, `readline()`, `readlines()`, `write()`, `writelines()`, `seek(pos)`, `tell()`, `close()`

10.2 Programs with files

(1) Count lines, words and characters

```
lines = words = chars = 0
with open("notes.txt") as f:
    for line in f:
        lines += 1
        words += len(line.split())
        chars += len(line)
print(lines, words, chars)
```

(2) Copy a file

```
import shutil
shutil.copy("notes.txt", "backup.txt")
```

(3) Find the most common word in a file

```
from collections import Counter
with open("notes.txt") as f:
    words = f.read().lower().split()
print(Counter(words).most_common(3))
```

10.3 Files and folders: os and pathlib

```
from pathlib import Path
import os

p = Path("data") / "report.txt"          # builds path for any OS
print(p.exists(), p.suffix, p.name, p.parent)
Path("data").mkdir(exist_ok=True)
for f in Path(".").glob("*.py"):         # all python files here
    print(f)
print(os.getcwd(), os.listdir("."))
os.remove("backup.txt")
```

10.4 Exceptions (run time errors)

An exception stops the program when something goes wrong. We can ***catch and handle*** *it so the program does not crash.*

```
try:
    n = int(input("Enter a number: "))
    print(10 / n)
except ValueError:
    print("Please enter digits only")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:                # any other error
    print("Unexpected:", e)
else:
    print("No error happened")        # runs only when no exception
finally:
    print("Always runs (cleanup)")    # close files, connections
```

Exception	When it happens
ValueError	right type, wrong value: `int("abc")`
TypeError	wrong type: `"a" + 1`
IndexError	list index out of range
KeyError	missing dictionary key
FileNotFoundError	file does not exist
ZeroDivisionError	divide by zero
AttributeError	object has no such attribute

Exception	When it happens
ImportError	module not found

10.5 Raising and custom exceptions

```
class InsufficientFunds(Exception):
    """Raised when balance is too low."""

def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise InsufficientFunds(f"Need {amount - balance} more")
    return balance - amount

try:
    withdraw(500, 800)
except InsufficientFunds as e:
    print("Error:", e)              # Error: Need 300 more
```

10.6 Safe file reading with exception handling

```
def read_file(path):
    try:
        with open(path, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"{path} not found")
    except PermissionError:
        print("No permission")
    return None
```

Industry use: Production code never lets a user error crash the whole system. It catches **specific** exceptions, logs them and continues or exits cleanly. Avoid bare `except:`.

MCQs - File Handling and Exception Handling

Q1. Best way to open a file is

(a) open() without close
(b) with open() as f
(c) file()
(d) read()

Q2. Mode "a" is used to

(a) read
(b) append
(c) delete
(d) copy

Q3. Mode "w" on existing file will

(a) append
(b) erase old content
(c) error
(d) read it

Q4. `f.readlines()` returns

(a) a string
(b) a list of lines
(c) a number
(d) a tuple

Q5. Code in `finally` runs

(a) only on error
(b) only without error
(c) always
(d) never

Q6. `int("abc")` raises

(a) TypeError
(b) ValueError
(c) KeyError
(d) IndexError

Q7. Which keyword throws an exception?

(a) throw
(b) raise
(c) error
(d) except

Q8. `d["x"]` on a missing key gives

(a) None
(b) KeyError
(c) 0
(d) ValueError

Q9. `else` in try-except runs when

(a) exception occurs
(b) no exception occurs
(c) always
(d) never

Q10. Which module gives easy path handling?

(a) pathlib
(b) paths
(c) filelib
(d) dir

Answers: 1-b 2-b 3-b 4-b 5-c 6-b 7-b 8-b 9-b 10-a

Chapter 11

Object Oriented Programming (OOP)

11.1 Why OOP?

OOP organizes a program around **objects** that keep **data (attributes)** and **actions (methods)** together. It makes big programs easier to build and maintain.

- **Class** - blueprint (like a house plan).
- **Object** - real thing made from the class (like a house).

11.2 Class and object

```
class Student:
    school = "ABC College"              # class variable (shared by all)

    def __init__(self, name, marks):    # constructor, runs on creation
        self.name = name                # instance variables (own copy)
        self.marks = marks

    def grade(self):                    # method
        return "Pass" if self.marks >= 40 else "Fail"

    def __str__(self):                  # how print() shows the object
        return f"{self.name} ({self.marks})"

s1 = Student("Sahil", 88)
s2 = Student("Amit", 35)
print(s1, s1.grade())        # Sahil (88) Pass
print(s2.grade())            # Fail
print(Student.school)
```

Remember: `self` means "this object". Python passes it automatically to every method as the first parameter.

11.3 Four pillars of OOP

Pillar	Meaning	Python example
Encapsulation	hide data, give controlled access	`_protected`, `__private`, `property`
Inheritance	child class reuses parent class	`class Dog(Animal)`
Polymorphism	same method name, different behaviour	`speak()` in Dog and Cat
Abstraction	show only what is needed	`abc.ABC`, abstract methods

11.4 Encapsulation

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance          # private (name mangling)

    def deposit(self, amt):
        if amt <= 0:
            raise ValueError("Invalid amount")
        self.__balance += amt

    def withdraw(self, amt):
        if amt > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amt

    @property
    def balance(self):                    # read only access
        return self.__balance

acc = BankAccount("Sahil", 1000)
acc.deposit(500)
print(acc.balance)        # 1500
# acc.balance = 5         # AttributeError (no setter)
```

11.5 Inheritance

```python
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):                    # Dog inherits Animal
    def speak(self):                  # method overriding
        return f"{self.name} says Woof"

class Cat(Animal):
    def __init__(self, name, color):
        super().__init__(name)        # call parent constructor
        self.color = color
    def speak(self):
        return f"{self.name} says Meow"

for pet in [Dog("Tommy"), Cat("Kitty", "white")]:
    print(pet.speak())                # polymorphism
```

```
Output:
Tommy says Woof
Kitty says Meow
```

Types of inheritance

Single (one parent), Multilevel (A -> B -> C), Multiple (`class C(A, B)`), Hierarchical (many children, one parent). Python decides method order using **MRO**: `C.mro()`.

11.6 Abstraction with abstract classes

```python
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2

class Rect(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h

shapes = [Circle(2), Rect(3, 4)]
print([round(s.area(), 2) for s in shapes])    # [12.57, 12]
# Shape()  -> TypeError: cannot create object of abstract class
```

11.7 Class methods and static methods

```python
class Employee:
    count = 0
    def __init__(self, name, salary):
        self.name, self.salary = name, salary
        Employee.count += 1

    @classmethod
    def from_string(cls, text):            # alternative constructor
        name, salary = text.split("-")
        return cls(name, int(salary))

    @staticmethod
    def is_valid_salary(x):                # no self, no cls: utility
        return x > 0

e = Employee.from_string("Ravi-50000")
print(e.name, e.salary, Employee.count, Employee.is_valid_salary(5))
```

MCQs - Object Oriented Programming (OOP)

Q1. Constructor in Python is

(a) __start__
(b) __init__
(c) __new_obj__
(d) constructor

Q2. `self` refers to

(a) the class
(b) the current object
(c) parent
(d) module

Q3. Inheritance means

(a) hiding data
(b) child class gets features of parent
(c) many methods same name in one class
(d) a loop

Q4. `super()` **is used to**

(a) delete object
(b) call parent class methods
(c) create class
(d) import

Q5. Overriding means

(a) same method in child replaces parent's
(b) two classes same name
(c) deleting method
(d) renaming

Q6. A private variable starts with

(a) _
(b) __
(c) #
(d) $

Q7. Which decorator makes a method read-only attribute?

(a) @staticmethod
(b) @property
(c) @classmethod
(d) @private

Q8. Abstract methods need which module?

(a) abs
(b) abc
(c) oop
(d) base

Q9. `class C(A, B)` **is**

(a) single inheritance
(b) multiple inheritance
(c) error
(d) abstraction

Q10. Object is

(a) blueprint
(b) instance of a class
(c) function
(d) module

Answers: 1-b 2-b 3-b 4-b 5-a 6-b 7-b 8-b 9-b 10-b

Chapter 12

Advanced OOP: Magic Methods and Dataclasses

12.1 Magic (dunder) methods

Special methods with double underscores let your objects behave like built-in types.

Method	Used for
`__init__`	create object
`__str__` / `__repr__`	print text / developer text
`__len__`	`len(obj)`
`__getitem__`	`obj[i]`
`__add__`, `__sub__`, `__mul__`	`+ - *` operators
`__eq__`, `__lt__`	`==`, `<` comparison
`__contains__`	`x in obj`
`__iter__` / `__next__`	loops
`__enter__` / `__exit__`	`with` statement
`__call__`	call object like function

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, o):  return Vector(self.x + o.x, self.y + o.y)
    def __eq__(self, o):   return (self.x, self.y) == (o.x, o.y)
    def __len__(self):     return 2
    def __repr__(self):    return f"Vector({self.x}, {self.y})"

v = Vector(1, 2) + Vector(3, 4)
print(v, v == Vector(4, 6), len(v))     # Vector(4, 6) True 2
```

12.2 Dataclasses (less code, same power)

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Product:
    name: str
    price: float
    tags: list = field(default_factory=list)

p1 = Product("Pen", 10.5)
p2 = Product("Book", 250)
print(p1)                    # Product(name='Pen', price=10.5, tags=[])
print(p1 < p2)               # order=True gives comparison
```

12.3 Composition ("has a") vs inheritance ("is a")

```
class Engine:
    def start(self): return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()          # Car HAS an Engine
    def start(self):
        return self.engine.start()
```

> **Tip:** Prefer **composition over inheritance** in large systems: it is more flexible and easier to test.

12.4 Context managers

```
class Timer:
    def __enter__(self):
        import time; self.t = time.time(); return self
    def __exit__(self, *exc):
        import time; print(f"Took {time.time() - self.t:.3f}s")

with Timer():
    sum(range(1_000_000))

from contextlib import contextmanager
@contextmanager
def opened(path):
    f = open(path);
    try: yield f
    finally: f.close()
```

12.5 Design patterns in Python

Singleton (one object only)

```
class Config:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst
print(Config() is Config())    # True
```

Factory (create object by name)

```
def make_shape(kind, *args):
    return {"circle": Circle, "rect": Rect}[kind](*args)
```

Observer (callbacks)

```
class Button:
    def __init__(self): self.handlers = []
    def on_click(self, fn): self.handlers.append(fn)
    def click(self):
        for h in self.handlers: h()
b = Button(); b.on_click(lambda: print("Clicked!")); b.click()
```

12.6 __slots__ and memory

`__slots__ = ("x", "y")` stops Python from creating a dictionary for every object. Saves memory when you create millions of objects.

MCQs - Advanced OOP: Magic Methods and Dataclasses

Q1. `__str__` is used for

(a) deleting
(b) printing readable text
(c) comparing
(d) importing

Q2. `__add__` is called for

(a) a - b
(b) a + b
(c) a * b
(d) a // b

Q3. `@dataclass` automatically creates

(a) __init__ and __repr__
(b) a database
(c) a file
(d) a thread

Q4. `__enter__` and `__exit__` belong to

(a) loops
(b) context manager
(c) lists
(d) tuples

Q5. "Car has an Engine" is

(a) inheritance
(b) composition
(c) abstraction
(d) polymorphism

Q6. Singleton pattern allows

(a) many objects
(b) only one object of class
(c) no objects
(d) static objects only

Q7. `__len__` is called by

(a) len(obj)
(b) size(obj)
(c) obj.len
(d) count(obj)

Q8. `__call__` makes an object

(a) callable like a function
(b) printable
(c) iterable
(d) private

Q9. `__eq__` supports

(a) =
(b) ==
(c) != only
(d) is

Q10. `__slots__` is used to

(a) save memory
(b) sort
(c) open files
(d) log

Answers: 1-b 2-b 3-a 4-b 5-b 6-b 7-a 8-a 9-b 10-a

Chapter 13

Iterators, Generators and Decorators

13.1 Iterables and iterators

An **iterable** can be looped over (list, str, dict ...). An **iterator** gives one item at a time using `next()`.

```
nums = [10, 20, 30]
it = iter(nums)
print(next(it), next(it), next(it))    # 10 20 30
# next(it) now raises StopIteration  (a for loop handles this for us)

class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1
        return self.n + 1
print(list(Countdown(3)))      # [3, 2, 1]
```

13.2 Generators (lazy values with yield)

A generator function uses `yield`. It produces values **one at a time** and remembers where it stopped, so it uses very little memory - perfect for huge data.

```
def fib_gen(limit):
    a, b = 0, 1
    while a < limit:
        yield a
        a, b = b, a + b

for x in fib_gen(50):
    print(x, end=" ")            # 0 1 1 2 3 5 8 13 21 34

squares = (x * x for x in range(1_000_000))   # generator expression
print(next(squares), next(squares))           # 0 1
print(sum(x * x for x in range(1000)))        # no big list created

def read_big_file(path):
    with open(path) as f:
        for line in f:
            yield line.strip()                # one line in memory at a time
```

List	Generator
all items stored in memory	creates item only when needed
can use many times	can be used only once
indexing allowed	no indexing

13.3 itertools and functional tools

```
import itertools as it
print(list(it.permutations("abc", 2)))        # [('a','b'),('a','c'),('b','a'),...]
print(list(it.combinations([1, 2, 3], 2)))    # [(1,2),(1,3),(2,3)]
print(list(it.product("ab", [1, 2])))         # [('a',1),('a',2),('b',1),('b',2)]
print(list(it.islice(it.count(10), 3)))       # [10, 11, 12]
from functools import lru_cache
@lru_cache(maxsize=None)                      # automatic caching (memoization)
def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(80))
```

13.4 Decorators

*A decorator **adds extra behaviour to a function** without changing its code. Functions are objects, so we can pass them to other functions.*

```
import time, functools

def timer(func):
    @functools.wraps(func)                # keeps original name and docstring
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
        return result
    return wrapper

@timer                                    # same as: slow = timer(slow)
def slow(n):
    return sum(range(n))

print(slow(5_000_000))
```

Decorator with arguments

```
def retry(times):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **kw):
            for attempt in range(1, times + 1):
                try:
                    return func(*a, **kw)
                except Exception as e:
                    print(f"Attempt {attempt} failed: {e}")
            raise RuntimeError("All retries failed")
        return wrapper
    return deco

@retry(3)
def connect():
    raise ConnectionError("server down")
```

***Industry use:** Decorators are used in web frameworks (`@app.route("/home")`), for login checks, caching, logging, timing and retry logic.*

13.5 Closures

```
def multiplier(n):
    def inner(x):
        return x * n          # inner remembers n
    return inner
double, triple = multiplier(2), multiplier(3)
print(double(5), triple(5))    # 10 15
```

13.6 Comprehension summary

```
[x for x in range(5)]            # list
{x for x in "hello"}             # set
{k: v for k, v in zip("ab", [1, 2])}   # dict
(x for x in range(5))            # generator (not a tuple!)
```

MCQs - Iterators, Generators and Decorators

Q1. Which keyword makes a function a generator?

(a) return
(b) yield
(c) gen
(d) lazy

Q2. `next()` on an exhausted iterator raises

(a) KeyError
(b) StopIteration
(c) EOFError
(d) None

Q3. Generators are good because they

(a) store all values
(b) save memory
(c) are faster than C
(d) can be indexed

Q4. `(x for x in range(3))` is a

(a) tuple
(b) list
(c) generator
(d) set

Q5. A decorator is applied with

(a) #
(b) @
(c) $
(d) &

Q6. `functools.wraps` **is used to**

(a) keep original function metadata
(b) delete function
(c) speed up
(d) open files

Q7. `lru_cache` **is used for**

(a) sorting
(b) caching results
(c) threading
(d) logging

Q8. A closure is

(a) an inner function that remembers outer variables
(b) a closed file
(c) end of loop
(d) a class

Q9. `list(itertools.combinations([1,2,3],2))` **length is**

(a) 3
(b) 6
(c) 9
(d) 2

Q10. A generator can be used

(a) many times
(b) once
(c) never
(d) only with lists

Answers: 1-b 2-b 3-b 4-c 5-b 6-a 7-b 8-a 9-a 10-b

Chapter 14

Working with Data: JSON, CSV, Dates and Database

14.1 JSON

JSON is the standard format for web APIs and configuration files.

```
import json
data = {"name": "Sahil", "skills": ["C", "Python"], "active": True, "score": None}

text = json.dumps(data, indent=2)        # dict -> JSON string
obj = json.loads(text)                   # JSON string -> dict

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)         # write to file
with open("data.json") as f:
    loaded = json.load(f)                # read from file
print(loaded["skills"][1])               # Python
```

Python	JSON
dict	object {}
list, tuple	array []
str	string
int, float	number
True / False / None	true / false / null

14.2 CSV files

```
import csv

with open("students.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["name", "marks"])
    w.writerows([["Sahil", 88], ["Amit", 76], ["Neha", 92]])

with open("students.csv") as f:
    rows = list(csv.DictReader(f))           # each row becomes a dict
print(rows[0])                               # {'name': 'Sahil', 'marks': '88'}
avg = sum(int(r["marks"]) for r in rows) / len(rows)
print(f"Average = {avg:.1f}")                # Average = 85.3
```

14.3 Date and time

```
from datetime import datetime, date, timedelta

now = datetime.now()
print(now.strftime("%d-%m-%Y %H:%M"))        # 25-10-2026 14:30
d = datetime.strptime("15-08-2004", "%d-%m-%Y")
print((date.today() - d.date()).days // 365, "years old")
print(date.today() + timedelta(days=30))     # date after 30 days
print(d.weekday(), d.strftime("%A"))         # day of week
```

Code	Meaning	Code	Meaning
%Y	year 2026	%m	month 01-12
%d	day 01-31	%H	hour 00-23
%M	minute	%S	second
%A	weekday name	%B	month name

14.4 SQLite database (built in)

```
import sqlite3
con = sqlite3.connect("school.db")
cur = con.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS student(id INTEGER PRIMARY KEY, name TEXT, marks INT)")
cur.execute("INSERT INTO student(name, marks) VALUES (?, ?)", ("Sahil", 88))   # ? protects from
cur.executemany("INSERT INTO student(name, marks) VALUES (?, ?)", [("Amit", 76), ("Neha", 92)])
con.commit()

for row in cur.execute("SELECT name, marks FROM student WHERE marks > ? ORDER BY marks DESC", (8
    print(row)               # ('Neha', 92)   ('Sahil', 88)
con.close()
```

Careful: Never build SQL by joining strings with user input (`"... WHERE name='" + x + "'"`). It allows **SQL injection**. Always use ? placeholders.

14.5 Pandas in 10 lines (data analysis)

```
import pandas as pd
df = pd.read_csv("students.csv")
print(df.head())
print(df["marks"].mean(), df["marks"].max())
print(df[df["marks"] > 80])                    # filter rows
df["grade"] = df["marks"].apply(lambda m: "A" if m >= 85 else "B")
print(df.sort_values("marks", ascending=False))
df.to_excel("result.xlsx", index=False)
```

14.6 Command line tools with argparse

```
import argparse
p = argparse.ArgumentParser(description="Word counter")
p.add_argument("file")
p.add_argument("--top", type=int, default=3, help="how many words")
args = p.parse_args()
print(args.file, args.top)
# python wc.py notes.txt --top 5
```

MCQs - Working with Data: JSON, CSV, Dates and Database

Q1. `json.loads()` **converts**

(a) dict to JSON
(b) JSON string to Python object
(c) file to dict
(d) CSV to JSON

Q2. `json.dumps()` **converts**

(a) JSON to dict
(b) Python object to JSON string
(c) file to list
(d) nothing

Q3. JSON `null` **becomes in Python**

(a) 0
(b) None
(c) ""
(d) False

Q4. `csv.DictReader` **returns each row as a**

(a) list
(b) dict
(c) tuple
(d) string

Q5. `strftime` **is used to**

(a) string to date
(b) date to formatted string
(c) sleep
(d) subtract

Q6. `timedelta(days=5)` **represents**

(a) a date
(b) a time difference
(c) a month
(d) a year

Q7. Which placeholder style protects SQL queries?

(a) string concatenation
(b) ? parameters
(c) f-string
(d) %s with input

Q8. `con.commit()` **does**

(a) closes connection
(b) saves changes to database
(c) deletes table
(d) connects

Q9. Library for data analysis tables

(a) pandas
(b) re
(c) random
(d) sys

Q10. `argparse` **is used for**

(a) command line options
(b) GUI
(c) database
(d) regex

Answers: 1-b 2-b 3-b 4-b 5-b 6-b 7-b 8-b 9-a 10-a

Chapter 15

Advanced Python for Industry

15.1 Logging (never use print in production)

```python
import logging
logging.basicConfig(
    filename="app.log", level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s")
log = logging.getLogger("payment")

log.debug("Detailed info for developers")
log.info("Payment started")
log.warning("Retrying...")
try:
    1 / 0
except ZeroDivisionError:
    log.exception("Payment failed")        # logs the full traceback
```

Level	Use
DEBUG	very detailed, for developers
INFO	normal events
WARNING	something unexpected, still working
ERROR	a task failed
CRITICAL	whole system in danger

15.2 Testing with pytest / unittest

```python
# calc.py
def divide(a, b):
    if b == 0:
        raise ValueError("b cannot be 0")
    return a / b

# test_calc.py   (run with:  pytest)
import pytest
from calc import divide

def test_divide():
    assert divide(10, 2) == 5

def test_zero():
    with pytest.raises(ValueError):
        divide(1, 0)

@pytest.mark.parametrize("a,b,r", [(8, 2, 4), (9, 3, 3), (5, 5, 1)])
def test_many(a, b, r):
    assert divide(a, b) == r
```

Industry use: Companies run automatic tests on every code change (**CI/CD**: GitHub Actions, Jenkins). Code without tests is risky to change.

15.3 Threads, processes and async

Tool	Best for	Notes
threading	waiting jobs (network, files)	GIL stops true parallel CPU work
multiprocessing	heavy CPU work	separate processes, real parallel
asyncio	thousands of network calls	single thread, `async` / `await`

```python
from concurrent.futures import ThreadPoolExecutor
import time

def download(n):
    time.sleep(1)                # pretend to wait for network
    return f"file{n} done"

with ThreadPoolExecutor(max_workers=4) as pool:
    results = list(pool.map(download, range(4)))
print(results)                   # takes ~1 second, not 4
```

```python
import asyncio

async def fetch(n):
    await asyncio.sleep(1)
    return n * 10

async def main():
    results = await asyncio.gather(*(fetch(i) for i in range(5)))
    print(results)               # [0, 10, 20, 30, 40] in ~1 second

asyncio.run(main())
```

Remember: GIL (Global Interpreter Lock): in standard CPython only one thread runs Python bytecode at a time. Use `multiprocessing` for CPU heavy work.

15.4 Thread safety

```python
import threading
counter, lock = 0, threading.Lock()
def work():
    global counter
    for _ in range(100000):
        with lock:               # protects shared data
            counter += 1
ts = [threading.Thread(target=work) for _ in range(4)]
[t.start() for t in ts]; [t.join() for t in ts]
print(counter)                   # 400000
```

15.5 A tiny web API with FastAPI

```
from fastapi import FastAPI
app = FastAPI()
items = {1: "pen", 2: "book"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"id": item_id, "name": items.get(item_id, "not found")}

# run:  uvicorn main:app --reload      then open  /docs
```

15.6 Performance tips

- Use **list/dict/set comprehensions** and built-ins (`sum`, `any`) - they are written in C.
- Use **set / dict for lookups** instead of searching a list.
- Use **generators** for large data; `"".join(list)` instead of `+=` in loops.
- Measure first: `timeit`, `cProfile`. Do not guess.

```
import timeit
print(timeit.timeit("sum(range(1000))", number=10000))
```

15.7 Clean code and standards

- Follow **PEP 8** (style): 4 spaces, `snake_case`, max ~79-88 chars per line. Tools: `black`, `flake8`, `ruff`.
- Write **docstrings** and **type hints**; keep functions small.
- Use `pathlib`, f-strings, `with`, `enumerate`, `zip` - "Pythonic" code.
- Keep secrets (passwords, API keys) in **environment variables**: `os.environ["API_KEY"]`, never in code.

MCQs - Advanced Python for Industry

Q1. Which level is used for normal events?

(a) DEBUG
(b) INFO
(c) CRITICAL
(d) ERROR

Q2. pytest checks results using

(a) assert
(b) print
(c) input
(d) return

Q3. For CPU heavy parallel work use

(a) threading
(b) multiprocessing
(c) logging
(d) json

Q4. GIL stands for

(a) General Input Loop
(b) Global Interpreter Lock
(c) Graph Item List
(d) Great Index Level

Q5. `await` is used inside

(a) normal function
(b) async function
(c) class only
(d) loop only

Q6. PEP 8 is a guide for

(a) memory
(b) code style
(c) networking
(d) databases

Q7. Protecting shared data in threads needs a

(a) Lock
(b) list
(c) tuple
(d) file

Q8. API keys should be stored in

(a) source code
(b) environment variables
(c) print statements
(d) comments

Q9. `ThreadPoolExecutor` is good for

(a) waiting (I/O) tasks
(b) heavy maths
(c) sorting
(d) typing

Q10. FastAPI automatically creates docs at

(a) /help
(b) /docs
(c) /info
(d) /about

Answers: 1-b 2-a 3-b 4-b 5-b 6-b 7-a 8-b 9-a 10-b

Chapter 16 Data Structures and Algorithms in Python

16.1 Stack (LIFO) using a list

```
stack = []
stack.append(10); stack.append(20); stack.append(30)    # push
print(stack.pop())       # 30  (last in, first out)
print(stack[-1])         # 20  (peek)
print(not stack)         # False  (is empty?)
```

16.2 Queue (FIFO) using deque

`list.pop(0)` is slow (O(n)). Use `collections.deque` for O(1) at both ends.

```
from collections import deque
q = deque()
q.append("A"); q.append("B"); q.append("C")    # enqueue
print(q.popleft())       # A  (first in, first out)
q.appendleft("Z")        # add at front
print(q)                 # deque(['Z', 'B', 'C'])
```

16.3 Linked list

```
class Node:
    def __init__(self, data):
        self.data, self.next = data, None

class LinkedList:
    def __init__(self):
        self.head = None
    def add_last(self, data):
        node = Node(data)
        if not self.head:
            self.head = node
            return
        cur = self.head
        while cur.next:
            cur = cur.next
        cur.next = node
    def reverse(self):
        prev, cur = None, self.head
        while cur:
            cur.next, prev, cur = prev, cur, cur.next
        self.head = prev
    def __iter__(self):
        cur = self.head
        while cur:
            yield cur.data
            cur = cur.next

ll = LinkedList()
for x in (1, 2, 3): ll.add_last(x)
ll.reverse()
print(list(ll))      # [3, 2, 1]
```

16.4 Binary search tree

```
class BST:
    def __init__(self, key):
        self.key, self.left, self.right = key, None, None
    def insert(self, k):
        if k < self.key:
            if self.left: self.left.insert(k)
            else: self.left = BST(k)
        elif k > self.key:
            if self.right: self.right.insert(k)
            else: self.right = BST(k)
    def inorder(self):
        if self.left:  yield from self.left.inorder()
        yield self.key
        if self.right: yield from self.right.inorder()

root = BST(50)
for k in (30, 70, 20, 40, 60): root.insert(k)
print(list(root.inorder()))    # [20, 30, 40, 50, 60, 70]
```

16.5 Graph: BFS and DFS

```python
from collections import deque
graph = {"A": ["B", "C"], "B": ["D"], "C": ["D", "E"], "D": [], "E": []}

def bfs(start):                        # level by level, uses queue
    seen, q, order = {start}, deque([start]), []
    while q:
        node = q.popleft(); order.append(node)
        for nb in graph[node]:
            if nb not in seen:
                seen.add(nb); q.append(nb)
    return order

def dfs(node, seen=None):              # deep first, uses recursion / stack
    seen = seen if seen is not None else set()
    seen.add(node)
    for nb in graph[node]:
        if nb not in seen: dfs(nb, seen)
    return seen

print(bfs("A"))      # ['A', 'B', 'C', 'D', 'E']
```

16.6 Sorting algorithms

```python
def quick_sort(a):
    if len(a) <= 1: return a
    p = a[len(a) // 2]
    return (quick_sort([x for x in a if x < p]) +
            [x for x in a if x == p] +
            quick_sort([x for x in a if x > p]))

def merge_sort(a):
    if len(a) <= 1: return a
    mid = len(a) // 2
    l, r = merge_sort(a[:mid]), merge_sort(a[mid:])
    out = []
    while l and r:
        out.append(l.pop(0) if l[0] <= r[0] else r.pop(0))
    return out + l + r

print(quick_sort([5, 2, 9, 1, 5, 6]), merge_sort([5, 2, 9, 1]))
```

Algorithm	Average	Worst	Note
Bubble / Insertion	O(n^2)	O(n^2)	simple, small data
Merge sort	O(n log n)	O(n log n)	stable, extra memory
Quick sort	O(n log n)	O(n^2)	fast in practice
Python `sorted()`	O(n log n)	O(n log n)	Timsort, stable

16.7 Heap and priority queue

```
import heapq
h = [5, 1, 8, 3]
heapq.heapify(h)
print(heapq.heappop(h))          # 1  (smallest first)
heapq.heappush(h, 0)
print(heapq.nsmallest(2, h))     # [0, 3]
print(heapq.nlargest(2, [4, 9, 1, 7]))   # [9, 7]
```

16.8 Dynamic programming idea

Save answers of small problems so they are not calculated again.

```
def coin_change(coins, amount):
    dp = [0] + [float("inf")] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1
print(coin_change([1, 5, 10, 25], 63))     # 6  (25+25+10+1+1+1)
```

MCQs - Data Structures and Algorithms in Python

Q1. Best Python structure for a fast queue

(a) list with pop(0)
(b) collections.deque
(c) tuple
(d) set

Q2. Stack is

(a) FIFO
(b) LIFO
(c) sorted
(d) random

Q3. Inorder traversal of a BST gives

(a) random order
(b) sorted order
(c) reverse
(d) level order

Q4. BFS uses

(a) stack
(b) queue
(c) heap
(d) set only

Q5. `heapq.heappop` **removes the**

(a) largest
(b) smallest
(c) last added
(d) random

Q6. Python's `sorted()` **algorithm is**

(a) bubble sort
(b) Timsort
(c) selection sort
(d) heap sort

Q7. Dictionary lookup time is

(a) O(n)
(b) O(1) average
(c) O(n^2)
(d) O(log n) always

Q8. Dynamic programming avoids

(a) loops
(b) repeating same calculation
(c) functions
(d) lists

Answers: 1-b 2-b 3-b 4-b 5-b 6-b 7-b 8-b

Chapter 17

NumPy and Matplotlib Basics

17.1 NumPy - fast numbers

NumPy gives the ndarray: a fixed-type array that works **10 to 100 times faster** than Python lists for maths. It is the base of Pandas, scikit-learn and TensorFlow.

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([[1, 2], [3, 4]])
print(a * 2, a + 10)          # [2 4 6 8] [11 12 13 14]  (no loop needed!)
print(a.mean(), a.sum(), a.std())
print(b.shape, b.T)           # (2, 2) and the transpose
print(b @ b)                  # matrix multiplication
print(np.arange(0, 10, 2))    # [0 2 4 6 8]
print(np.linspace(0, 1, 5))   # [0. 0.25 0.5 0.75 1.]
print(np.zeros((2, 3)), np.ones(3), np.eye(2))
```

Indexing and filtering

```
x = np.array([5, 12, 7, 20, 3])
print(x[x > 6])                # [12  7 20]   boolean mask
print(np.where(x > 6, 1, 0))   # [0 1 1 1 0]
print(x.reshape(5, 1).shape)   # (5, 1)
m = np.random.default_rng(1).integers(1, 10, size=(3, 3))
print(m[1, :], m[:, 2])        # row 1 and column 2
```

17.2 Matplotlib - draw graphs

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
plt.plot(x, np.sin(x), label="sin")
plt.plot(x, np.cos(x), label="cos", linestyle="--")
plt.title("Sine and Cosine")
plt.xlabel("Angle"); plt.ylabel("Value")
plt.legend(); plt.grid(True)
plt.savefig("wave.png", dpi=150)
plt.show()
```

Chart	Function	Use
Line	plt.plot	trends over time
Bar	plt.bar	compare categories
Histogram	plt.hist	distribution of values
Scatter	plt.scatter	relation between 2 numbers
Pie	plt.pie	share of a whole

17.3 Mini project: marks analysis

```
import numpy as np, matplotlib.pyplot as plt
subjects = ["Maths", "Physics", "Chem", "English", "Python"]
marks = np.array([78, 85, 69, 74, 92])
print("Average:", marks.mean(), "Best:", subjects[marks.argmax()])
colors = ["tab:green" if m >= marks.mean() else "tab:red" for m in marks]
plt.bar(subjects, marks, color=colors)
plt.axhline(marks.mean(), color="black", linestyle=":")
plt.title("Marks of Sahil"); plt.ylim(0, 100)
plt.savefig("marks.png")
```

17.4 Pandas quick table

Task	Code
Read file	`df = pd.read_csv("a.csv")`
First rows / info	`df.head()`, `df.info()`, `df.describe()`
Select column / rows	`df["age"]`, `df[df["age"] > 18]`
Missing values	`df.isna().sum()`, `df.fillna(0)`, `df.dropna()`
New column	`df["total"] = df["a"] + df["b"]`
Group and summarize	`df.groupby("city")["sales"].sum()`
Join tables	`pd.merge(df1, df2, on="id")`
Save	`df.to_csv("out.csv", index=False)`

MCQs - NumPy and Matplotlib Basics

Q1. NumPy main object is

(a) list
(b) ndarray
(c) dict
(d) tuple

Q2. `np.array([1,2,3]) * 2` **gives**

(a) [1,2,3,1,2,3]
(b) [2 4 6]
(c) error
(d) 6

Q3. Which library draws graphs?

(a) numpy
(b) matplotlib
(c) re
(d) sys

Q4. `plt.bar` **makes a**

(a) line chart
(b) bar chart
(c) pie chart
(d) table

Q5. `df.isna().sum()` **shows**

(a) mean
(b) count of missing values
(c) rows
(d) columns

Q6. `@` **between two arrays means**

(a) bitwise and
(b) matrix multiplication
(c) email
(d) decorator

Q7. Boolean mask `x[x > 6]` **returns**

(a) all items
(b) items greater than 6
(c) index values
(d) error

Q8. NumPy is faster than lists because

(a) written in C and uses fixed type memory
(b) uses internet
(c) is shorter
(d) uses GPU always

Answers: 1-b 2-b 3-b 4-b 5-b 6-b 7-b 8-a

Chapter 18

Industry Oriented Programs

Tip: These programs are small versions of real tasks done by Python developers: **automation, data processing, APIs, testing and monitoring**. Type them, run them, then improve them.

Program 1: Bank Account System (OOP + exceptions + JSON storage)

Used in: banking, billing and wallet software.

```
import json
from pathlib import Path

class InsufficientFunds(Exception):
    pass

class Account:
    def __init__(self, acc_no, owner, balance=0.0):
        self.acc_no, self.owner, self._balance = acc_no, owner, balance
        self.history = []

    @property
    def balance(self):
        return self._balance

    def deposit(self, amt):
        if amt <= 0:
            raise ValueError("Amount must be positive")
        self._balance += amt
        self.history.append(("deposit", amt))

    def withdraw(self, amt):
        if amt > self._balance:
            raise InsufficientFunds(f"Balance {self._balance}, need {amt}")
        self._balance -= amt
        self.history.append(("withdraw", amt))

    def to_dict(self):
        return {"acc_no": self.acc_no, "owner": self.owner,
                "balance": self._balance, "history": self.history}

class Bank:
    def __init__(self, path="bank.json"):
        self.path, self.accounts = Path(path), {}
        if self.path.exists():
            for d in json.loads(self.path.read_text()):
                a = Account(d["acc_no"], d["owner"], d["balance"])
                a.history = d["history"]
                self.accounts[a.acc_no] = a
```

```
    def open(self, acc_no, owner, amount=0):
        self.accounts[acc_no] = Account(acc_no, owner, amount)

    def transfer(self, src, dst, amt):
        self.accounts[src].withdraw(amt)
        self.accounts[dst].deposit(amt)

    def save(self):
        self.path.write_text(json.dumps([a.to_dict() for a in self.accounts.values()], indent=2)

bank = Bank()
bank.open(1, "Sahil", 5000); bank.open(2, "Amit", 1000)
try:
    bank.transfer(1, 2, 1500)
    bank.transfer(2, 1, 9999)
except InsufficientFunds as e:
    print("Transfer failed:", e)
bank.save()
print(bank.accounts[1].balance, bank.accounts[2].balance)    # 3500 2500
```

Program 2: Log file analyzer (regex + Counter)

Used in: DevOps, security monitoring, support teams.

```
import re
from collections import Counter

LOG = re.compile(r"(?P<ip>\d+\.\d+\.\d+\.\d+) .* \[(?P<time>.*?)\] \"(?P<method>\w+) (?P<url>\S+

def analyze(path):
    ips, errors, urls = Counter(), Counter(), Counter()
    with open(path) as f:
        for line in f:
            m = LOG.search(line)
            if not m:
                continue
            ips[m["ip"]] += 1
            urls[m["url"]] += 1
            if m["status"].startswith(("4", "5")):
                errors[m["status"]] += 1
    return ips.most_common(3), urls.most_common(3), errors

top_ips, top_urls, errs = analyze("access.log")
print("Top visitors:", top_ips)
print("Error codes :", dict(errs))
```

Program 3: Automatic file organizer (automation)

Used in: office automation, backup scripts.

```
from pathlib import Path
import shutil

RULES = {".jpg": "Images", ".png": "Images", ".pdf": "Documents",
         ".docx": "Documents", ".mp4": "Videos", ".py": "Code", ".zip": "Archives"}

def organize(folder):
    folder = Path(folder)
    for item in folder.iterdir():
```

```
        if item.is_file():
            target = folder / RULES.get(item.suffix.lower(), "Others")
            target.mkdir(exist_ok=True)
            shutil.move(str(item), target / item.name)
            print(f"Moved {item.name} -> {target.name}/")

organize("Downloads")
```

Program 4: Sales report with pandas

Used in: data analysis, business reporting.

```
import pandas as pd
df = pd.read_csv("sales.csv", parse_dates=["date"])    # columns: date, region, product, qty, pr
df["revenue"] = df["qty"] * df["price"]

by_region = df.groupby("region")["revenue"].sum().sort_values(ascending=False)
monthly = df.groupby(df["date"].dt.to_period("M"))["revenue"].sum()
top_products = df.groupby("product")["qty"].sum().nlargest(5)

with pd.ExcelWriter("report.xlsx") as xl:
    by_region.to_excel(xl, sheet_name="Region")
    monthly.to_excel(xl, sheet_name="Monthly")
    top_products.to_excel(xl, sheet_name="TopProducts")
print(by_region.head())
```

Program 5: Weather / API client with retry and cache

Used in: any app that talks to outside services.

```
import requests, time, json
from functools import lru_cache

def get_json(url, retries=3, timeout=5):
    for attempt in range(1, retries + 1):
        try:
            r = requests.get(url, timeout=timeout)
            r.raise_for_status()              # error for 4xx / 5xx
            return r.json()
        except requests.RequestException as e:
            print(f"Attempt {attempt} failed: {e}")
            time.sleep(2 ** attempt)          # wait 2, 4, 8 sec (exponential backoff)
    raise RuntimeError("Service unavailable")

@lru_cache(maxsize=32)
def weather(city):
    url = f"https://wttr.in/{city}?format=j1"
    data = get_json(url)
    return data["current_condition"][0]["temp_C"]

print(weather("Pune"))
```

Program 6: Employee management with SQLite

Used in: small internal tools and back-end services.

```
import sqlite3
from contextlib import closing

class EmployeeDB:
```

```
    def __init__(self, path="emp.db"):
        self.con = sqlite3.connect(path)
        self.con.execute("""CREATE TABLE IF NOT EXISTS emp(
            id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,
            dept TEXT, salary REAL CHECK(salary >= 0))""")

    def add(self, name, dept, salary):
        with self.con:                                  # auto commit / rollback
            self.con.execute("INSERT INTO emp(name,dept,salary) VALUES (?,?,?)", (name, dept, sa

    def dept_report(self):
        with closing(self.con.cursor()) as cur:
            cur.execute("SELECT dept, COUNT(*), AVG(salary) FROM emp GROUP BY dept")
            return cur.fetchall()

db = EmployeeDB(":memory:")
db.add("Ravi", "IT", 50000); db.add("Neha", "IT", 60000); db.add("Amit", "HR", 40000)
print(db.dept_report())      # [('HR', 1, 40000.0), ('IT', 2, 55000.0)]
```

Program 7: Simple REST API (FastAPI) with validation

Used in: back-end for mobile and web applications.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()
class Task(BaseModel):
    title: str = Field(min_length=1, max_length=100)
    done: bool = False

tasks: dict[int, Task] = {}
next_id = 1

@app.post("/tasks", status_code=201)
def add_task(task: Task):
    global next_id
    tasks[next_id] = task
    next_id += 1
    return {"id": next_id - 1, **task.model_dump()}

@app.get("/tasks/{tid}")
def get_task(tid: int):
    if tid not in tasks:
        raise HTTPException(404, "Task not found")
    return tasks[tid]
```

Program 8: Web scraping basics (requests + BeautifulSoup)

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://example.com", timeout=10).text
soup = BeautifulSoup(html, "html.parser")
print(soup.title.text)
for a in soup.find_all("a"):
    print(a.get("href"), a.text.strip())
```

Remember: Always read a website's **terms and robots.txt** before scraping, and do not send too many requests.

Program 9: Machine learning in 10 lines (scikit-learn)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)
model = RandomForestClassifier(n_estimators=100).fit(X_tr, y_tr)
print("Accuracy:", accuracy_score(y_te, model.predict(X_te)))   # about 0.97 to 1.0
```

Program 10: Password strength checker and generator

Used in: security tools and sign-up forms.

```
import re, secrets, string

def strength(pw):
    score = sum([len(pw) >= 12, bool(re.search(r"[a-z]", pw)), bool(re.search(r"[A-Z]", pw)),
                 bool(re.search(r"\d", pw)), bool(re.search(r"[^\w\s]", pw))])
    return ["Very weak", "Weak", "Fair", "Good", "Strong", "Excellent"][score]

def generate(n=14):
    chars = string.ascii_letters + string.digits + "!@#$%&*"
    return "".join(secrets.choice(chars) for _ in range(n))   # secrets, not random!

print(strength("hello"), "|", strength("Sahil@2026Py!"))   # Very weak | Excellent
print(generate())
```

MCQs - Industry Oriented Programs

Q1. `raise_for_status()` **in requests**

(a) raises an error on bad HTTP status
(b) saves file
(c) sorts data
(d) prints status

Q2. Exponential backoff means

(a) retry immediately
(b) wait longer after each failure
(c) never retry
(d) retry at same speed

Q3. `groupby` **in pandas is used to**

(a) group rows and summarize
(b) open file
(c) sort alphabetically only
(d) delete rows

Q4. For secure random passwords use

(a) random
(b) secrets
(c) time
(d) math

Q5. `with self.con:` **in sqlite3 does**

(a) auto commit or rollback
(b) closes Python
(c) deletes db
(d) nothing

Q6. pydantic is mostly used for

(a) data validation
(b) drawing
(c) sound
(d) sorting

Q7. `Path.iterdir()` **returns**

(a) items in a folder
(b) file size
(c) current time
(d) parent

Q8. Before web scraping you should check

(a) robots.txt and terms
(b) CPU speed
(c) font
(d) nothing

Answers: 1-a 2-b 3-a 4-b 5-a 6-a 7-a 8-a

Chapter 19

Practice Programs

Tip: Try each program **without looking** at the notes. Level: (E) easy, (M) medium, (H) hard. Write them in your notebook, then run them on the computer.

A. Basic programs

- 1. (E) Print your name, class and city using f-string.
- 2. (E) Read two numbers and print sum, difference, product, quotient and remainder.
- 3. (E) Area and perimeter of a circle, rectangle and triangle.
- 4. (E) Convert Celsius to Fahrenheit and kilometres to miles.
- 5. (E) Check if a number is even or odd; positive, negative or zero.
- 6. (E) Largest of three numbers (with and without `max()`).
- 7. (M) Convert seconds to hh:mm:ss.
- 8. (M) Electricity bill: first 100 units Rs 3, next 100 Rs 5, above Rs 8.
- 9. (M) Quadratic equation roots (use `cmath` for complex roots).
- 10. (M) Simple calculator using `match-case`.

B. Loops and patterns

- 11. (E) Multiplication table of a number.
- 12. (E) Sum of first N numbers and sum of squares.
- 13. (M) Prime numbers between 1 and 200.
- 14. (M) Perfect numbers, Armstrong numbers, palindromic numbers up to 1000.
- 15. (M) Fibonacci series using loop, recursion and generator.
- 16. (M) Decimal to binary / octal / hex without `bin()`.
- 17. (M) GCD and LCM (try `math.gcd`).
- 18. (M) Print these patterns:

```
A               1 2 3 4 5       *
A B             1 2 3 4         * *
A B C           1 2 3           * * *
A B C D         1 2             * *
                1               *
```

- 19. (H) Pascal's triangle and number pyramid.

- 20. (H) Number guessing game with hints and score.

C. Strings, lists, dictionaries

- 21. (E) Count vowels, consonants, digits and spaces in a string.
- 22. (E) Reverse a string and each word of a sentence.
- 23. (M) Check pangram ("The quick brown fox jumps over the lazy dog").
- 24. (M) Find the longest word and the most frequent word.
- 25. (M) Compress a string: `aaabbc -> a3b2c1`.
- 26. (M) Find the first non-repeating character.
- 27. (M) Second largest, remove duplicates, rotate a list.
- 28. (M) Merge two sorted lists without using `sort()`.
- 29. (M) Matrix addition, multiplication, transpose using lists.
- 30. (M) Student result: dictionary of name -> marks, print topper, average, grades.
- 31. (M) Group anagrams: `["eat","tea","tan","ate","nat"]`.
- 32. (H) Longest substring without repeating characters.
- 33. (H) Valid brackets using a stack list: `"{[()]}"`.
- 34. (H) Spiral print of a matrix.

D. Functions, OOP, files

- 35. (E) Function to check prime; use it in a list comprehension.
- 36. (M) Recursive: power, sum of digits, binary search, Tower of Hanoi.
- 37. (M) Decorator that prints the time taken by a function.
- 38. (M) Generator that gives prime numbers forever.
- 39. (M) Class `Rectangle` with area, perimeter, `__str__`, `__eq__`.
- 40. (M) `BankAccount` with deposit, withdraw and minimum balance rule.
- 41. (M) `Shape` abstract class with `Circle`, `Square`, `Triangle`.
- 42. (M) Library system: classes `Book`, `Member`, `Library` with issue and return.
- 43. (M) Write 10 numbers to a file, read them back and print sum and average.
- 44. (M) Count lines, words, characters of a text file; find the top 5 words.
- 45. (H) Contact book saved in JSON with add, search, edit, delete.
- 46. (H) Implement a Stack, a Queue and a linked list as classes.
- 47. (H) Binary search tree class with insert, search, inorder.

E. Industry mini projects

Project	Concepts used
To-do list app (command line)	list, dict, JSON, functions
Expense tracker with monthly report	CSV, datetime, pandas, matplotlib
Student management with SQLite	OOP, sqlite3, exceptions
Quiz game with timer and scores	random, time, files
Email / SMS reminder bot	smtplib, schedule, datetime
Weather app using free API	requests, JSON, error handling
Web scraper for product prices	requests, BeautifulSoup, CSV
Chat server using sockets	socket, threading
URL shortener API	FastAPI, dict / SQLite, hashing
Face / object detection demo	OpenCV, numpy

Sample solution: Compress string

```
def compress(s):
    if not s:
        return ""
    out, count = [], 1
    for prev, cur in zip(s, s[1:]):
        if cur == prev:
            count += 1
        else:
            out.append(f"{prev}{count}")
            count = 1
    out.append(f"{s[-1]}{count}")
    return "".join(out)
print(compress("aaabbc"))     # a3b2c1
```

Sample solution: Valid brackets

```
def valid(s):
    pairs, stack = {")": "(", "]": "[", "}": "{"}, []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
print(valid("{[()]}"), valid("{[(])}"))     # True False
```

Sample solution: Group anagrams

```
from collections import defaultdict
def group(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())
print(group(["eat", "tea", "tan", "ate", "nat"]))
# [['eat','tea','ate'], ['tan','nat']]
```

Chapter 20

Interview Questions and Answers

1. What is the difference between list and tuple?

List is mutable and written with `[ ]`. Tuple is immutable, written with `( )`, faster and can be a dictionary key.

2. What is the difference between `==` and `is`?

`==` compares values. `is` checks if both names point to the **same object** in memory.

3. What are mutable and immutable types?

Mutable (list, dict, set) can change after creation. Immutable (int, str, tuple) cannot; changing makes a new object.

4. What is `*args` and `**kwargs`?

`*args` collects extra positional arguments in a tuple, `**kwargs` collects extra keyword arguments in a dict.

5. What is a decorator? A generator?

Decorator wraps a function to add behaviour. Generator uses `yield` to produce values lazily one by one.

6. Shallow copy vs deep copy?

Shallow copy copies the outer object only; inner objects are shared. Deep copy (`copy.deepcopy`) copies everything inside too.

7. What is the GIL?

A lock in CPython that lets only one thread execute Python bytecode at a time. Use multiprocessing for CPU parallelism; threads are fine for waiting (I/O) work.

8. How does Python manage memory?

Reference counting plus a cyclic **garbage collector**. Objects are freed when nothing refers to them.

9. What is `__init__` vs `__new__`?

`__new__` creates the object, `__init__` sets it up after creation.

10. What are Python's namespaces and scope?

Local, Enclosing, Global, Built-in (LEGB). `global` and `nonlocal` change outer variables.

11. What is the difference between `append` and `extend`?

`append(x)` adds x as one item; `extend(list)` adds every item of the list.

`[1].append([2,3])` gives `[1,[2,3]]`, `extend` gives `[1,2,3]`.

12. What is a lambda function?

A small anonymous one-line function: `lambda x: x * 2`.

13. What is the difference between a module and a package?

Module = one `.py` file. Package = folder of modules with `__init__.py`.

14. What is the use of `if __name__ == "__main__"`?

Runs code only when the file is executed directly, not when imported.

15. How do you handle exceptions? What is `finally`?

`try / except / else / finally`. `finally` always runs and is used for cleanup.

16. What is duck typing?

"If it walks like a duck and quacks like a duck, it is a duck." Python checks what an object **can do**, not its class name.

17. Output questions (famous tricks)

```
print(0.1 + 0.2 == 0.3)        # False (float precision) -> use math.isclose
print([] == False, bool([]))   # False False... [] is falsy but not equal to False
a = [1, 2, 3]; b = a; b += [4]; print(a)   # [1, 2, 3, 4]  (same list)
print(type(1 / 2), 7 // -2)    # <class 'float'> -4  (floor division rounds down)
def f(x=[]): x.append(1); return x
print(f(), f())                # [1, 1] [1, 1]  (shared default list!)
print("abc"[5:])               # '' (slicing never gives IndexError)
```

Interview tips

- Practise **coding on paper / whiteboard**: reverse, palindrome, two-sum, anagram, FizzBuzz, factorial, Fibonacci, linked list.
- Explain **time complexity**: list search O(n), dict / set lookup O(1), sort O(n log n).
- Know OOP, decorators, generators, exceptions, file handling, and one framework (Django / Flask / FastAPI) or one data library (pandas).
- Keep a **GitHub profile** with 3 or 4 good projects.

Special Section

Quick Revision Cheat Sheet

Syntax at a glance

```
name = "Sahil"; nums = [1, 2, 3]; info = {"a": 1}; pts = (1, 2); uniq = {1, 2}

def add(a, b=0, *args, **kw): return a + b

class Dog(Animal):
    def __init__(self, n): super().__init__(n)

for i, x in enumerate(nums): print(i, x)
squares = [x * x for x in nums if x > 1]
try:
    risky()
except (ValueError, KeyError) as e:
    print(e)
finally:
    cleanup()
with open("f.txt") as f: data = f.read()
```

Key points to remember

Topic	Remember
Indentation	4 spaces, defines blocks
Types	int float str bool list tuple set dict None
Mutable	list, dict, set; immutable: int, str, tuple
/ vs //	true division vs floor division
Slicing	`s[start:stop:step]`, stop not included
== vs is	value vs same object
Mutable default	never use `def f(x=[])`
Strings	f-strings are best: `f"{x:.2f}"`
Files	always use `with open(...)`
Errors	catch specific exceptions, avoid bare `except`
OOP	class, object, self, `__init__`, inheritance, `super()`
Environment	`python -m venv venv`, `pip install`, `requirements.txt`

Common methods

Type	Methods
str	upper lower strip split join replace find count startswith format
list	append extend insert remove pop sort reverse index count copy
dict	get keys values items update pop setdefault
set	add remove discard union intersection difference

Learning path after these notes

- 1. Solve 50 programs from the practice section and 100 problems on LeetCode / HackerRank.
- 2. Learn **Data Structures and Algorithms** in Python.
- 3. Choose a path: **Web** (Django / FastAPI), **Data / AI** (NumPy, Pandas, scikit-learn), **Automation / DevOps** (scripts, Docker, cloud).
- 4. Learn Git, Linux commands, SQL and testing (pytest).
- 5. Build 3 real projects and publish them on GitHub.

Remember: Code every day for 30 minutes. Reading alone does not make a programmer - **typing and running** does.

Thank you for reading!

Keep coding, keep practising -
every bug teaches you something.

Sahil

Notes prepared by Sahil